CSS STYLING, LAYOUTS, FLEXBOX AND GRID

A practical, illustrated learning book for BSIT students

Original explanations • CSS code examples • Visual layout diagrams

Contents

Chapter 1: CSS Foundations

1.1 What CSS Does

1.2 Selectors and Specificity

1.3 The Box Model

1.4 Display and Position

1.5 Units and Responsive Sizing

Chapter 2: Flexbox Fundamentals

2.1 Flex Containers and Items

2.2 Direction and Axes

2.3 Justification and Alignment

2.4 Wrapping and Gaps

2.5 Grow, Shrink, and Basis

2.6 Order and Accessibility

Chapter 3: CSS Grid Fundamentals

3.1 Grid Containers and Tracks

3.2 Fractional Units and Repeat

3.3 Explicit and Implicit Grids

3.4 Placing and Spanning Items

3.5 Grid Areas

3.6 Auto-Fit and Auto-Fill

Chapter 4: Combining Layout Systems

4.1 Choosing Flexbox or Grid

4.2 Responsive Navigation

4.3 Cards and Media Objects

4.4 Sticky Headers and Sidebars

4.5 Images in Flexible Layouts

Chapter 5: Responsive Web Design

5.1 Mobile-First Design

5.2 Media Queries

5.3 Container Queries

5.4 Fluid Typography and Spacing

5.5 Overflow and Debugging

Chapter 6: Practical Website Layouts

6.1 Two-Column Article Page

6.2 Dashboard Layout

6.3 Photo and Book Gallery

6.4 Form Layout

6.5 Production Checklist

Chapter 7: Advanced CSS Layout Practices

7.1 Subgrid

7.2 Logical Properties

7.3 Alignment and Intrinsic Sizing

7.4 Motion and Reduced Motion

7.5 Maintainable CSS Architecture

7.6 Flexbox and Grid Review

Chapter 1 — CSS Foundations

1.1 What CSS Does

CSS controls the visual presentation of HTML content. A rule selects elements and assigns declarations, such as color, spacing, and typography. Separating structure from presentation makes a site easier to update and maintain.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
h1 { color: #194c75; font-size: 2rem; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 1.1 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Start with semantic HTML, then add a stylesheet. Change one declaration at a time and inspect the result in a browser.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

1.2 Selectors and Specificity

Type selectors target elements, class selectors target reusable groups, and ID selectors target unique identifiers. Specificity helps determine which competing declaration wins, but cascade origin, importance, and source order also matter.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.card { padding: 1rem; }
.card.featured { border: 2px solid navy; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 1.2 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Prefer reusable classes instead of deeply nested selectors. Use developer tools to inspect which declarations are applied.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

1.3 The Box Model

Every element has content, padding, border, and margin. Padding adds space inside the border; margin separates the element from neighbors. The border-box sizing model includes padding and borders within a declared width.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
* { box-sizing: border-box; }
.panel { width: 320px; padding: 20px; border: 1px solid; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 1.3 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

A width of 320px stays 320px with border-box. Without it, horizontal padding and borders add to the rendered width.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

1.4 Display and Position

Normal flow lays out block and inline content. The display property can create block, inline, flex, or grid formatting contexts. Positioning is useful for overlays, but it should not replace normal layout for an entire page.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.notice { position: sticky; top: 0; }
.badge { position: absolute; right: 8px; top: 8px; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 1.4 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

A positioned badge needs an appropriate positioned ancestor. Sticky positioning also depends on scroll containers.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

1.5 Units and Responsive Sizing

Pixels provide fixed CSS lengths, rem relates to the root font size, percentages depend on a containing size, and viewport units relate to the viewport. Modern functions such as min(), max(), and clamp() support flexible sizing.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.page { width: min(92%, 1100px); margin: auto; }
h1 { font-size: clamp(1.6rem, 4vw, 3rem); }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 1.5 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Test large text settings and narrow screens. Flexible sizing should preserve readability rather than force everything to shrink.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

Chapter 2 — Flexbox Fundamentals

2.1 Flex Containers and Items

Flexbox is a one-dimensional layout system. A flex container controls the distribution and alignment of its direct children along a main axis and a cross axis. It works especially well for toolbars, navigation, and small card groups.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.toolbar { display: flex; gap: 12px; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 2.1 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Only direct children become flex items. A nested element needs its own display:flex rule if its children should also participate.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

2.2 Direction and Axes

flex-direction determines the main axis. Row is horizontal in a typical left-to-right writing mode; column is vertical. The cross axis runs perpendicular to the main axis. Axis direction matters when choosing alignment properties.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.stack { display: flex; flex-direction: column; }
.inline { display: flex; flex-direction: row; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 2.2 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Draw an arrow for the main axis before choosing justify-content or align-items. This prevents common alignment mistakes.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

2.3 Justification and Alignment

justify-content distributes available space on the main axis. align-items aligns items on the cross axis. align-content distributes flex lines when wrapping creates multiple lines and there is extra cross-axis space.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.nav { display: flex; justify-content: space-between; align-items: center; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 2.3 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

For vertically stacked items, justify-content affects vertical distribution. Use gap for consistent spacing between siblings.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

2.4 Wrapping and Gaps

Flex items normally share one line. flex-wrap allows additional lines when there is not enough space. Gap creates consistent space between items without requiring margins on each child.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.chips { display: flex; flex-wrap: wrap; gap: .75rem; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 2.4 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Wrapping does not guarantee aligned columns. Use Grid when consistent two-dimensional tracks are important.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

2.5 Grow, Shrink, and Basis

flex-grow allocates positive free space, flex-shrink controls contraction, and flex-basis supplies an initial main size. The shorthand flex: 1 is often useful for equal distribution but min-width may prevent expected shrinking.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.sidebar { flex: 0 0 220px; }
.content { flex: 1 1 0; min-width: 0; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 2.5 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Set min-width:0 on flexible content that contains long URLs or code to avoid unwanted overflow.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

2.6 Order and Accessibility

The order property changes visual placement, not document order. Keyboard navigation and screen readers generally follow source order. Avoid using order to create a visual sequence that conflicts with the reading sequence.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.item--highlight { order: 2; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 2.6 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Place meaningful content in logical HTML order first. Use visual reordering only when it does not confuse navigation.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

Chapter 3 — CSS Grid Fundamentals

3.1 Grid Containers and Tracks

Grid is a two-dimensional layout model that controls rows and columns. A grid container establishes tracks; its direct children become grid items. It is suited to page shells, galleries, and repeated card layouts.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.gallery { display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 16px; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 3.1 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Unlike Flexbox, Grid can align items to shared column and row boundaries.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

3.2 Fractional Units and Repeat

The fr unit shares available track space. repeat() reduces repetitive declarations. minmax() sets bounds for tracks and is particularly useful for flexible card grids.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.cards { display: grid; grid-template-columns: repeat(3, minmax(0, 1fr)); }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 3.2 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

The minmax(0,1fr) pattern allows columns to shrink even when content has a large intrinsic width.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

3.3 Explicit and Implicit Grids

Explicit tracks are defined by grid-template-columns and grid-template-rows. Extra items can create implicit tracks. grid-auto-rows and grid-auto-columns influence those additional tracks.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.gallery { grid-template-columns: repeat(3, 1fr); grid-auto-rows: minmax(140px, auto); }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 3.3 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

A gallery with seven cards and three columns creates an additional row automatically.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

3.4 Placing and Spanning Items

Grid lines can place items precisely. grid-column and grid-row accept start and end positions or span counts. Spanning is useful for featured cards, sidebars, and hero sections.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.feature { grid-column: span 2; }
.tall { grid-row: span 2; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 3.4 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Check small screens carefully: a two-column span needs a layout with at least two columns.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

3.5 Grid Areas

Named grid areas make complex page structures easier to read. A template can describe header, sidebar, main content, and footer in a visual arrangement.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.layout { display:grid; grid-template-areas: "head head" "side main" "foot foot"; }
header { grid-area: head; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 3.5 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Each named area must form a rectangle in the template. Use a dot for intentionally empty cells.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

3.6 Auto-Fit and Auto-Fill

repeat(auto-fit, minmax(...)) can create responsive tracks without fixed breakpoints. auto-fit collapses empty repeated tracks, while auto-fill preserves their space. Both can be useful for card collections.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.cards { display:grid; grid-template-columns:repeat(auto-fit,minmax(220px,1fr)); gap:1rem; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 3.6 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Try both keywords when the container has only one or two cards. Their behavior differs most when extra columns would fit.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

Chapter 4 — Combining Layout Systems

4.1 Choosing Flexbox or Grid

Use Flexbox when arranging a row or column of items with content-driven sizes. Use Grid when columns and rows must align across multiple items. These are complementary tools, not competing replacements.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.page { display:grid; grid-template-columns: 240px 1fr; }
.menu { display:flex; gap:1rem; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 4.1 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

A typical site uses Grid for the overall page and Flexbox for navigation and button groups.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

4.2 Responsive Navigation

A navigation bar may stack on narrow screens and align horizontally when space is available. Semantic nav and list elements provide a useful foundation; layout properties handle presentation.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.links { display:flex; flex-wrap:wrap; gap:1rem; }
@media (max-width:600px) { .links { flex-direction:column; } }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 4.2 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Keep links accessible by keyboard. Avoid hiding navigation without providing an operable alternative.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

4.3 Cards and Media Objects

A card grid can use Grid for consistent columns, while each card uses Flexbox to stack its title, description, and action. This combination keeps actions aligned even when descriptions vary in length.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.cards { display:grid; grid-template-columns:repeat(auto-fit,minmax(240px,1fr)); }
.card { display:flex; flex-direction:column; }
.card a { margin-top:auto; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 4.3 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

The auto top margin pushes the action toward the bottom of a flex column.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

4.4 Sticky Headers and Sidebars

Sticky positioning can keep important controls visible as users scroll. It must be applied carefully because a large sticky element can obscure content on small screens.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.sidebar { position:sticky; top:1rem; align-self:start; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 4.4 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Avoid making tall sidebars sticky on short viewports. Test zoom and mobile orientations.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

4.5 Images in Flexible Layouts

Responsive images should not overflow their containers. Set max-width:100% and height:auto for intrinsic scaling; object-fit can crop images when a fixed aspect ratio is desired.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
img { max-width:100%; height:auto; }
.card img { width:100%; aspect-ratio:16/9; object-fit:cover; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 4.5 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Always include meaningful alt text for informative images and empty alt text for purely decorative images.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

Chapter 5 — Responsive Web Design

5.1 Mobile-First Design

Mobile-first styling begins with a simple single-column layout and enhances it as available space grows. This helps prioritize content and reduces the number of overrides needed for narrow viewports.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.page { display:block; }
@media (min-width:800px) { .page { display:grid; grid-template-columns:240px 1fr; } }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 5.1 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Choose breakpoints based on where content becomes cramped rather than assuming exact device categories.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

5.2 Media Queries

Media queries apply styles conditionally based on viewport or device features. Width queries are common, but orientation, pointer capability, and user preferences can also guide presentation.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
@media (max-width:640px) { .hero { padding:1.5rem; } }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 5.2 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Resize the browser gradually and watch for overflow or uncomfortable line lengths.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

5.3 Container Queries

Container queries allow a component to respond to its containing element rather than only the viewport. This is useful for reusable cards and widgets that appear in sidebars or main content areas.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.widget-wrap { container-type:inline-size; }
@container (min-width:420px) { .widget { display:flex; } }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 5.3 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

A component can adapt differently in a narrow sidebar and a wide content column on the same screen.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

5.4 Fluid Typography and Spacing

Fluid design uses flexible values that remain within readable limits. clamp() can set a minimum, preferred, and maximum size for headings, gaps, and padding.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
h1 { font-size:clamp(1.8rem,3vw,3.4rem); }
section { padding:clamp(1rem,3vw,3rem); }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 5.4 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Avoid using viewport units alone for body text because text may become too small.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

5.5 Overflow and Debugging

Horizontal scrolling often comes from fixed widths, long strings, oversized images, or flex items with intrinsic minimum sizes. Browser developer tools reveal the offending element and computed styles.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.content { min-width:0; overflow-wrap:anywhere; }
img { max-width:100%; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 5.5 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Do not use overflow-x:hidden as the first fix; it can conceal broken layouts instead of solving them.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

Chapter 6 — Practical Website Layouts

6.1 Two-Column Article Page

A reading layout often includes a table of contents and a main article. Grid can create a fixed-width sidebar and a flexible article, then collapse to one column on small screens.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.article-layout { display:grid; grid-template-columns:220px minmax(0,1fr); gap:2rem; }
@media(max-width:700px){.article-layout{grid-template-columns:1fr;}}
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 6.1 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Limit paragraph line length with max-width to keep long educational text comfortable to read.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

6.2 Dashboard Layout

A dashboard may contain a header, sidebar, summary cards, and data tables. Use Grid for the outer shell and the card region; use Flexbox inside individual cards for labels and values.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.stats { display:grid; grid-template-columns:repeat(auto-fit,minmax(180px,1fr)); gap:1rem; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 6.2 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Wide tables may need a dedicated horizontal scroll region rather than shrinking all columns until text is unreadable.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

6.3 Photo and Book Gallery

A gallery can use repeat(auto-fit,minmax()) for responsive columns. Set consistent image aspect ratios, keep captions readable, and avoid stretching low-resolution cover art.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.books { display:grid; grid-template-columns:repeat(auto-fit,minmax(170px,1fr)); gap:1.25rem; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 6.3 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Store book cover images separately so they can be replaced without changing the page structure.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

6.4 Form Layout

Forms need clear labels, predictable spacing, and controls that are large enough to operate. Grid can align fields on wide screens while a single-column layout works well on phones.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.form-grid { display:grid; grid-template-columns:repeat(2,minmax(0,1fr)); gap:1rem; }
@media(max-width:650px){.form-grid{grid-template-columns:1fr;}}
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 6.4 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Never rely only on placeholder text as a field label. Provide visible labels and clear error messages.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

6.5 Production Checklist

Before publishing, inspect layouts at multiple widths, zoom levels, and text sizes. Check keyboard order, focus indicators, image alternatives, overflow, contrast, and reduced-motion preferences.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.button:focus-visible { outline:3px solid #174d79; outline-offset:3px; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 6.5 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Test real content: long titles, empty states, multiple paragraphs, and large data sets expose issues that sample text can hide.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

Chapter 7 — Advanced CSS Layout Practices

7.1 Subgrid

Subgrid lets nested grids align selected tracks with a parent grid. It is useful when cards need titles, descriptions, and footers to share consistent row lines across neighboring cards.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.cards { display:grid; grid-template-columns:repeat(3,1fr); }
.card { display:grid; grid-template-rows:subgrid; grid-row:span 3; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 7.1 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Use subgrid when nested content must share the parent track geometry rather than creating independent rows.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

7.2 Logical Properties

Logical properties describe spacing and dimensions relative to writing direction. They support layouts in both left-to-right and right-to-left languages without hard-coding physical left and right sides.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.panel { padding-inline:1.5rem; margin-block:1rem; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 7.2 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Prefer margin-inline-start to margin-left when the design should adapt to different writing directions.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

7.3 Alignment and Intrinsic Sizing

Intrinsic sizing keywords such as min-content, max-content, and fit-content let content influence track sizes. Combining them with minmax() gives designers more control over flexible columns.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.shell { display:grid; grid-template-columns:fit-content(260px) minmax(0,1fr); }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 7.3 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Watch for long unbreakable strings, which can still force unexpected widths unless overflow is handled.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

7.4 Motion and Reduced Motion

Transitions can help communicate state changes, but motion should not be necessary to understand a page. Respect reduced-motion preferences for users who find animation uncomfortable.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.card { transition:transform .2s ease; }
@media(prefers-reduced-motion:reduce){.card{transition:none;}}
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 7.4 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

Keep interactive feedback visible even when animation is disabled.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

7.5 Maintainable CSS Architecture

Organize styles around components and shared design tokens. Use custom properties for consistent spacing, colors, and widths. Avoid overly specific selectors and duplicate declarations.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
:root { --space:1rem; --brand:#205b82; }
.card { padding:var(--space); border-color:var(--brand); }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 7.5 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

A small, consistent naming system makes future changes easier across multiple pages.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.

7.6 Flexbox and Grid Review

A strong layout starts with meaningful HTML, then chooses Flexbox or Grid according to the relationship among elements. Responsiveness, accessibility, and testing are part of the layout process—not final decorations.

Concept in practice

A useful way to understand this feature is to begin with a small component, predict how it will appear, and then modify one CSS declaration. Browser developer tools make the effect visible immediately. The following original example demonstrates the main technique for this topic.

```
.shell { display:grid; gap:1rem; }
.actions { display:flex; flex-wrap:wrap; gap:.5rem; }
```

Reading the example

The selector identifies the HTML elements to style. Each declaration changes a specific aspect of layout or presentation. Compare the result with the default browser layout, then adjust the values to understand how the browser distributes available space.



Figure 7.6 — Layout learning model. The browser combines document structure and CSS rules to produce the displayed result. This is an original replaceable conceptual figure.

Practical guidance

For each component, ask whether it needs one-dimensional flow or shared two-dimensional alignment.

Try changing the example in a local HTML file and observing the behavior at different browser widths. Check that text remains readable and that controls retain a sensible order when the layout changes.